

# Minimum Time to Reach Destination

Authored by: Arpit Raj, LNMIIT Jaipur | July 24, 2026

## Visual Problem Solution Approach

This problem is modeled as a shortest-path traversal on a 2D grid with dynamic edge weights. We use a **Modified Dijkstra's Algorithm** utilizing a Priority Queue to always expand the path taking the minimum time.

### Core Logic & Waiting Time Calculations:

- When evaluating a neighboring cell  $(nr, nc)$ , we check the minimum arrival time: `moveTime[nr][nc]`.
- If `currdiss < moveTime[nr][nc]`:  
*"we are using currdiss, kyunki ho skta pohochne se pehle gate band"*  
This implies we reached the cell before it is available to enter.
- *"we will wait jab tak room khul na jaaye"*  
Therefore, our effective arrival time synchronizes to the room's opening time:  
`moveTime[nr][nc]`.
- *"aur phir 1 second to move in"*  
So, the updated time becomes: `newdiss = moveTime[nr][nc] + 1`.

If we arrive when the room is already open, it simply takes 1 second to move in: `newdiss = currdiss + 1`. This guarantees we track the optimal waiting time and travel duration for each step.

## Source Code Implementation

```
class Solution {
public:
    int minTimeToReach(vector<vector<int>>& moveTime) {

        int n=moveTime.size();
        int m=moveTime[0].size();

        vector<pair<int,int>> dir={{-1,0},{0,-1},{1,0},{0,1}};

        priority_queue<pair<long long,pair<int,int>>,vector<pair<long long,pair<int,int>>>,greater<pair<long long,pair<int,int>>>> pq;

        vector<vector<long long>> dist(n,vector<long long>(m,LLONG_MAX));
        pq.push({0,{0,0}});
        dist[0][0]=0;

        while(!pq.empty()){
            int r=pq.top().second.first;
            int c=pq.top().second.second;
            long long currdiss=pq.top().first;

            pq.pop();
            if(currdiss>dist[r][c]) continue;
            if(r==n-1 && c==m-1) return currdiss;

            for(auto &it:dir){

                int nr=r+it.first;
                int nc=c+it.second;
                if(nr>=0 && nc>=0 && nr<n && nc<m){
                    long long newdiss=currdiss+1;
                    if(currdiss<moveTime[nr][nc]){ //we are using
currdiss , kyunki ho skta pohochne se pehle gate band
newdiss=moveTime[nr][nc]+1; //we will wait jab tak
room khul na jaaye
//aur phir 1 second to
move in
                    }

                    if(newdiss<dist[nr][nc]){
                        dist[nr][nc]=newdiss;
                        pq.push({newdiss,{nr,nc}});
                    }
                }
            }
        }
    }
}
```

```
        return -1;

    }

};
```